INSTITUTO FEDERAL DE MINAS GERAIS – *CAMPUS ITABIRITO*
CURSO TÉCNICO INTEGRADO EM AUTOMAÇÃO INDUSTRIAL

ALISSANDERSON FELIPE BARROS NAZARETH

GABRIEL PEREIRA CLÓVIS

MIGUEL HENRIQUE DE AGUIAR SANTOS

ROBÔ MÓVEL GUIADO POR QR CODE (GMR)

Transporte automatizado utilizando Raspberry Pi e visão computacional

ITABIRITO
JANEIRO DE 2026

ALISSANDERSON FELIPE BARROS NAZARETH
GABRIEL PEREIRA CLÓVIS
MIGUEL HENRIQUE DE AGUIAR SANTOS

ROBÔ MÓVEL GUIADO POR QR CODE (GMR)

Transporte automatizado utilizando Raspberry Pi e visão computacional

Trabalho de conclusão de curso apresentado ao Curso Técnico Integrado em Automação Industrial do Instituto Federal de Minas Gerais, *Campus* Itabirito, como requisito parcial para obtenção do título de Técnico em Automação Industrial.

Orientador: Ms. Elton Verkalterem Reis Silva
Orientador: Dr. Taciana Almeida Garrido De Resende

Banca Examinadora:
Prof. Ms. Elton Verkalterem Reis Silva (IFMG) – Orientador 1
Prof. Dr. Taciana Almeida Garrido De Resende (IFMG) - Orientador 2
Prof. Dr. Diego Augusto Gonzaga (IFMG)
Prof. Ms. Thiago De Sousa Goveia (IFMG)

Trabalho de Conclusão de Curso defendido e aprovado em Itabirito, 30 de Janeiro de 2026.

ITABIRITO
JANEIRO DE 2026

AGRADECIMENTOS

Primeiramente, agradecemos a Deus, por nos conceder força, sabedoria e perseverança ao longo de toda essa jornada, tornando possível a realização deste trabalho.

Agradecemos também às nossas famílias, que sempre estiveram ao nosso lado, nos apoiando, incentivando e contribuindo para a nossa formação pessoal e acadêmica, moldando-nos para sermos quem somos hoje.

Expressamos nossa sincera gratidão aos nossos orientadores, Taciana Almeida Garrido e Elton Verkalterem Reis Silva, pelo apoio fundamental no desenvolvimento deste projeto, pelos conhecimentos compartilhados, pelas orientações precisas e pelo auxílio técnico, incluindo a disponibilização de peças essenciais, como o Raspberry Pi.

Agradecemos ao servidor Leandro Henrique Vidigal Sousa e à Raissa Rúbia Braga Gonçalves, pela disponibilidade do ambiente de inovação, pelo apoio oferecido, pela cessão de peças e materiais, bem como pelo uso dos computadores e da infraestrutura desse espaço, que foram de grande importância para a execução do trabalho.

Estendemos nossos agradecimentos a todos os demais funcionários e servidores da instituição, pelo suporte e pela manutenção do ambiente acadêmico, pois sem o trabalho deles este projeto também não seria possível.

Aos professores, nosso reconhecimento pelos conhecimentos transmitidos ao longo do curso e pela capacitação proporcionada, que contribuíram diretamente para nossa formação e para a realização deste trabalho.

Aos amigos e colegas que nos acompanharam durante toda essa trajetória, agradecemos pelo companheirismo, apoio e incentivo nos momentos de dificuldade.

Por fim, agradecemos ao Instituto Federal de Minas Gerais (IFMG) e ao curso de Automação Industrial, pela oportunidade de aprendizado, crescimento e desenvolvimento acadêmico e profissional.

ROBÔ MÓVEL GUIADO POR QR CODE (GMR)

Transporte automatizado utilizando Raspberry Pi e visão computacional

Alissanderson Felipe Barros Nazareth, Gabriel Pereira Clóvis e
Miguel Henrique de Aguiar Santos

Orientadores: Elton Verkalterem Reis Silva e Taciana Almeida Garrido De Resende

Resumo. Este trabalho apresenta o desenvolvimento de um robô móvel guiado por marcadores visuais (QR Codes), a fim de servir como base para uma plataforma móvel de transporte, voltada ao estudo de navegação automatizada de baixo custo. A proposta consiste na utilização de códigos QR contendo instruções de movimentação, permitindo que o robô execute deslocamentos entre pontos pré-determinados a partir de uma posição inicial conhecida. O sistema utiliza um microcomputador Raspberry Pi 3 como unidade de controle central, associado a uma câmera e utilizando a biblioteca de programação OpenCV para a leitura e interpretação dos códigos QR. O robô é capaz de identificar os marcadores visuais e executar comandos de movimento correspondentes, demonstrando a viabilidade da abordagem proposta como prova de conceito para sistemas de navegação baseada em visão computacional. Entretanto, o robô enfrenta dificuldades como baixa precisão de movimento, o que limita sua aplicação em terrenos irregulares, com muitas curvas ou deslocamentos extensos. Essas limitações evidenciam os desafios envolvidos no desenvolvimento de soluções de transporte automatizado que demandam maior precisão, estabilidade e confiabilidade. Os resultados obtidos confirmam a possibilidade de integração entre visão computacional e sistemas embarcados para navegação automatizada, ao mesmo tempo em que destacam a necessidade de aprimoramentos mecânicos e de controle para aplicações práticas futuras. O projeto contribui como base experimental para estudos em automação, robótica móvel e sistemas de navegação de baixo custo.

Palavras-chave: Plataforma móvel; Robótica; Robô guiado; Raspberry Pi; QR Code; Visão computacional.

ROBÔ MÓVEL GUIADO POR QR CODE (GMR)
Transporte automatizado utilizando Raspberry Pi e visão computacional

Alissanderson Felipe Barros Nazareth, Gabriel Pereira Clóvis e
Miguel Henrique de Aguiar Santos

Orientadores: Elton Verkalterem Reis Silva e Taciana Almeida Garrido De Resende

Abstract. This work presents the development of a mobile robot guided by visual markers (QR codes), intended to serve as a basis for a mobile transport platform aimed at the study of low-cost automated navigation. The proposed approach consists of using QR codes containing movement instructions, allowing the robot to travel between predetermined points from a known initial position. The system uses a Raspberry Pi 3 microcomputer as the central control unit, combined with a camera and the OpenCV programming library for reading and interpreting QR codes. The robot is capable of identifying visual markers and executing the corresponding motion commands, demonstrating the feasibility of the proposed approach as a proof of concept for computer vision–based navigation systems. However, the robot does not have a physical platform for transporting objects and exhibits low movement accuracy, which limits its application on uneven terrain, in environments with many curves, or over long distances. These limitations highlight the challenges involved in developing automated transport solutions that require greater precision, stability, and reliability. The results confirm the possibility of integrating computer vision and embedded systems for automated navigation, while also emphasizing the need for mechanical and control improvements for future practical applications. The project contributes as an experimental basis for studies in automation, mobile robotics, and low-cost navigation systems.

Keywords: Mobile platform; Robotics; Guided Robot; Raspberry Pi; QR code, Computer vision.

Lista de ilustrações

Figura 1 – Motor DC 3-6V com Caixa de Redução e Eixo Duplo 5
Figura 2 – Módulo sensor óptico 6
Figura 3 – Disco do encoder 6
Figura 4 – Webcam Logitech C920 6
Figura 5 – Raspberry Pi 3 Model B 7
Figura 6 – Módulo Driver Ponte H - L298N 7
Figura 7 – Kit Carregador Duplo+4 Baterias 18650 3,7v 7800mah 8
Figura 8 – Módulo controlador de tensão 8
Figura 9 – Estrutura de acrílico e roda boba 9
Figura 10 – Vista lateral GMR 10
Figura 11 – Fluxograma do funcionamento do sistema 11
Figura 12 – Representação da string de movimento 14
Figura 13 – Imagem dos setores para o teste realizado 16
Figura 14 – Vista em perspectiva GMR (sem baterias) 16
Figura 15 – Vista frontal GMR (sem baterias) 16

Lista de Tabelas

Tabela 1 – Custos estimados do projeto 18

Lista de Abreviaturas e Siglas

IFMG - Instituto Federal de Minas Gerais

GMR – *Guided Mobile Robot*

QR CODE - *Quick Response* Code

MDF – *Medium Density Fiberboard*

OPENCV - *Open Source Computer Vision Library*

PWM - *Pulse Width Modulation*

DC/CC - Corrente Contínua/*Direct Current*

FPS - Frames per Second

mAH - Miliampere-Hora

Led - *Light Emitting Diode*

3D - Tridimensional

RFID - *Radio Frequency Identification*

LiDAR - *Light Detection and Ranging*

AGVs - *Automated Guided Vehicles*

AMRs - *Autonomous Mobile Robots*

Sumário

1 - INTRODUÇÃO 1
2 - OBJETIVOS 2
2.1 - *Objetivo Geral* 2
2.2 - *Objetivos Específicos* 2
3 - REVISÃO DA LITERATURA 2
4 - METODOLOGIA 4
5 - RESULTADOS 16
6 - CONSIDERAÇÕES FINAIS 18
7 - REFERÊNCIAS 21
APÊNDICE A - ESTRUTURA E FUNCIONAMENTO 25
APÊNDICE B - CÓDIGO-FONTE E REPOSITÓRIO DO PROJETO 26

1 INTRODUÇÃO

Ao longo da história, diversas soluções foram desenvolvidas com o objetivo de facilitar o transporte de objetos. A partir do século XIX, com o surgimento dos motores a combustão interna, tornou-se possível mecanizar o deslocamento de cargas de maior massa, ampliando a capacidade produtiva de diferentes setores. Posteriormente, o desenvolvimento dos motores elétricos possibilitou o avanço da automação, permitindo maior controle e eficiência nos sistemas de transporte (LANCE DAY, 1996).

Nesse contexto, esteiras transportadoras acionadas por motores elétricos consolidaram-se como padrão industrial. No século XX, a introdução de robôs industriais marcou um novo estágio da automação, destacando-se o Unimate, considerado o primeiro robô industrial, adotado inicialmente pela General Motors para operações repetitivas e de risco (UNIVERSAL ROBOTS BRASIL, 2023).

No século XXI, os sistemas de transporte automatizado evoluíram significativamente, incorporando maior autonomia e flexibilidade. Dentre as soluções mais difundidas destacam-se os AGVs (*Automated Guided Vehicles*) e os AMRs (*Autonomous Mobile Robots*), amplamente utilizados no transporte interno de cargas. Enquanto os AGVs dependem de trilhas físicas ou fitas magnéticas para navegação, os AMRs utilizam sensores como LIDAR para mapear o ambiente e definir rotas de forma autônoma (Wang, 2025).

Apesar das vantagens, essas soluções apresentam elevado custo de implementação e baixa flexibilidade para alterações de rota, o que dificulta sua adoção em ambientes educacionais ou de menor escala. Diante disso, surgem propostas que utilizam sensores alternativos, como câmeras e marcadores visuais, visando reduzir custos e simplificar a configuração de trajetos.

Nesse cenário, este trabalho propõe o desenvolvimento de uma plataforma móvel de baixo custo, capaz de transportar objetos entre pontos pré-determinados por meio da leitura de marcadores QR, permitindo a criação e modificação de rotas de forma simples e acessível.

2 OBJETIVOS

2.1 Objetivo Geral

Desenvolver um recurso didático em miniatura, capaz de executar tarefas de forma guiada ou controlada, servindo como modelo funcional para aplicações educativas e experimentais.

2.2 Objetivos Específicos

- Realizar a leitura dos marcadores visuais utilizando a biblioteca OpenCV, garantindo que o robô possa identificar sua posição no ambiente e receber instruções.
- Projetar a base para uma plataforma transportadora
- Criar um aplicativo de comando, permitindo controlar e programar a movimentação da plataforma.
- Projetar e construir o chassi da plataforma, garantindo estabilidade e resistência para o transporte dos objetos.

3 REVISÃO DA LITERATURA

Para um robô de transporte, o método usado para traçar o caminho até um determinado ponto é uma decisão de suma importância. Foram analisados diferentes métodos de navegação para definir a melhor estratégia a ser aplicada de forma a atender os objetivos propostos. Para aplicações de maior precisão e automação, o método utilizando sensores LiDAR se tornam uma ótima opção. LiDAR, sigla para “light detection and ranging” (detecção e alcance por luz), é uma tecnologia de sensoriamento remoto que usa feixes de laser para medir distâncias e movimentos com precisão, em tempo real (IBM, 2022). Porém, optar por esse tipo de sensoriamento implica em um aumento significativo no custo do robô e na complexidade de seu projeto. Observadas essas dificuldades, consideramos o uso de sensores ultrassônicos, que utilizam ondas sonoras para saber a distância de um objeto ou superfície. Apesar da sua disponibilidade no contexto do desenvolvimento deste trabalho, o seu uso individual se mostrou insuficiente por não ser capaz de diferenciar com clareza os obstáculos identificados pelo sensor, dificultando a definição de uma referência padrão (as paredes), resultando em variações de comportamento. Além disso, estes sensores, se aplicados exclusivamente,

dificultam a identificação da localização e consequentemente a tomada de decisão. Para a proposta deste trabalho as melhores soluções apresentadas foram o uso de marcadores visuais, nesse caso, os QR codes, ou *tags* RFID. O RFID é um sistema que utiliza emissão e recepção de frequências de rádio, e oferece a leitura de dados de forma menos dependente de um campo de visão limitado, como o das câmeras, além de apresentar um custo relativamente baixo (IBM, 2018). Entretanto, no contexto de desenvolvimento pensado, faria-se necessário uma quantidade numerosa de *tags*, resultando em um preço final mais custoso. Além disso, a necessidade de uso de receptores de radiofrequência aumenta significativamente a quantidade de variáveis de entrada, acarretando um controle mais complexo e exigindo uma rede de comunicação entre receptores e o controlador. Estes obstáculos dificultaram o desenvolvimento e podem atrapalhar a proposta acadêmica com alunos principiantes. Esses fatores foram decisivos ao optar pelo método da movimentação guiada por meio da leitura de QR Codes, utilizando a visão computacional.

A visão computacional consiste em uma área da computação que permite aos sistemas interpretar, processar e extrair informações relevantes a partir de imagens e vídeos, com o objetivo de compreender o ambiente e tomar decisões com base nesses dados. Utilizando-a para a leitura de QR Codes, é possível receber dados para comandos de movimentação, bem como identificar a localização atual do robô (Lee; Hyun; Park, 2013), sem exigir custos elevados, dependendo apenas de uma câmera de qualidade razoável, o que permite mais flexibilidade e adaptabilidade para atender às premissas propostas. Essa abordagem possibilita o controle automatizado e favorece o aprendizado prático de conceitos como processamento de imagem, lógica de decisão e integração entre hardware e software (Vieira et al., 2024).

O trabalho de Gabriel Conceição, Letícia Colombo e Lucas Ferreira (2019) sobre uma empilhadeira seguidora de linha serviu como ponto de partida para o desenvolvimento deste projeto, uma vez que a proposta apresentava diversas semelhanças com o sistema aqui desenvolvido. Entre os aspectos em comum, destacam-se a geração de sinal PWM, o emprego de drivers de motor e a estrutura com componentes impressos em 3D, ainda que no nosso projeto a impressão 3D tenha sido utilizada apenas para o módulo das baterias. Outro ponto considerado foi uma sugestão presente nas considerações finais do referido trabalho, que propunha a substituição dos três microcontroladores ESP 32 por um microcomputador Raspberry Pi para suportar o processamento de dados recebidos dos sensores.

Seguindo essa linha de aprimoramento, adotamos o Raspberry Pi como unidade principal de controle, visando melhorar o desempenho geral do robô.

A escolha dos marcadores visuais (QR Codes) como tecnologia de localização e comunicação do GMR baseou-se principalmente em seu baixo custo, facilidade de implementação e precisão adequada para aplicações experimentais. Outras tecnologias de posicionamento incluem sensores ultrassônicos, LiDAR, GPS, RFID, ondas de rádio e sistemas de visão com inteligência artificial, cada uma com diferentes níveis de precisão, custo e complexidade. Por exemplo, LiDAR e sistemas de visão avançados oferecem alta precisão, mas demandam maior poder computacional e recursos financeiros; enquanto ultrassom e RFID são mais baratos, porém menos precisos e mais suscetíveis a interferências. Apesar disso, testes práticos são necessários para confirmar o desempenho da plataforma móvel em comparação com as medições relatadas na literatura (Semborski; Idzkowski, 2024).

4 METODOLOGIA

Neste capítulo serão descritos os procedimentos adotados para o desenvolvimento da plataforma robótica, a qual deve ser compreendida como um recurso didático, voltado à experimentação e ao desenvolvimento de conhecimentos técnicos. O projeto caracteriza-se como uma pesquisa aplicada, com abordagem experimental, sendo realizado principalmente nas dependências da instituição. Ao longo desta seção, são apresentadas as etapas gerais de concepção, montagem dos componentes e desenvolvimento da programação.

4.1 Materiais

Nas seções seguintes, são apresentados e detalhados os principais componentes utilizados na concepção e montagem do protótipo.

4.1.1 Motores

O projeto possui dois motores, responsáveis pela locomoção do sistema. Ambos são motores DC 3–6V com caixa de redução e eixo duplo conforme apresentado na figura 1.

amplamente utilizados em projetos de robótica por apresentarem bom torque, baixo consumo e dimensões compactas (GLOBALSPEC, 2024). Esses motores são responsáveis pela movimentação das rodas da plataforma, permitindo deslocamentos lineares e curvas por meio do controle independente de cada lado, caracterizando um sistema de tração diferencial.

Figura 1 – Motor DC 3-6V com Caixa de Redução e Eixo Duplo

Fonte: RoboCore (2026)

4.1.2 Sensores

O módulo sensor óptico observado na figura 2, associado ao disco de fendas (figura 3) constitui o encoder, um sensor utilizado para medir a rotação e de eixos ou rodas. Ele funciona por meio de um emissor e um receptor de luz infravermelha, que, por meio de circuitos eletrônicos, altera o nível lógico na saída do módulo no momento em que ocorre uma obstrução na passagem do feixe de luz, nesse caso, realizada pelas seções do disco, que é acoplado ao eixo do motor. Cada interrupção do feixe gera uma alteração fixa na tensão da saída, denominada “nível lógico”, sendo o nível lógico baixo associado à tensão comum (GND), e o nível lógico alto a uma tensão positiva. Essa alteração de tensão, quando associada à programação, permite ao sistema calcular a rotação, e, portanto, a velocidade ou o deslocamento do robô.

O robô utiliza dois encoders, com um disco com 20 fendas mostrado no figura 3, permitindo até 40 alterações de nível lógico a cada rotação completa da roda, possibilitando o monitoramento e controle do movimento da plataforma.

A câmera Logitech C920, apresentada na Figura 4, foi utilizada no projeto para a detecção de marcadores visuais (QR Codes), auxiliando no processo de navegação do robô. Com resolução de 1920x1080 pixels e taxa de atualização de imagem de 30 fps, a câmera é capaz de capturar imagens nítidas durante a etapa de leitura de QR Codes. Seu foco automático e amplo campo de visão contribuem para uma leitura mais eficiente dos QR Codes

em diferentes condições de posicionamento. Conectada ao Raspberry Pi por meio de interface USB plug and play, a câmera fornece as informações necessárias ao sistema de controle, colaborando diretamente para a execução dos comandos de movimentação.

Figura 2 – Módulo sensor óptico

Fonte: Mercado livre (2026)

Figura 3 – Disco do encoder

Fonte: Mercado livre (2026)

Figura 4 – Webcam Logitech C920

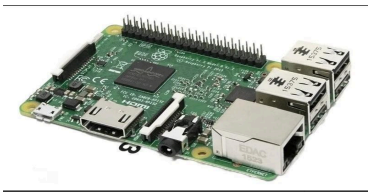
Fonte: Amazon (2026)

4.1.3 Microcomputador

O sistema da plataforma conta com um microcomputador, o Raspberry Pi 3, observado na figura 5. Ele é responsável pelo processamento de informações e comunicação dos componentes. Ele recebe dados da câmera e dos encoders e, por meio da programação, processa-os e os usa para coordenar a movimentação do GMR (o robô) se comunicando com

o driver apresentado na figura 6.

Figura 5 – Raspberry Pi 3 Model B

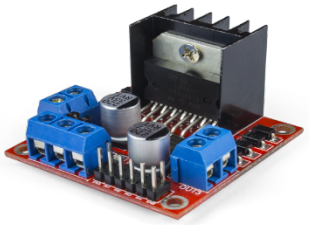


Fonte: Mercado livre (2026)

4.1.4 Driver

O driver usado no GMR, o Módulo L298N (figura 6), um tipo de circuito comumente chamado de “ponte H”, é um componente essencial para o controle dos motores DC. A ponte H recebe sinais de controle de baixa corrente (em torno de 20 mA), que gerenciam o acionamento dos motores. Com a capacidade para controlar até 2 motores individualmente, ela permite definir o sentido de rotação (horário ou anti horário), o que permite que os motores girem para frente ou para trás. O chip de circuito integrado do módulo, o L298N, suporta uma velocidade de comutação rápida, o que permite o controle de velocidade dos motores através do sinal PWM enviado pelo Raspberry Pi. O módulo permite a alimentação de fontes distintas entre o circuito lógico do módulo e a carga, ou seja, é possível alimentar o circuito de carga usando uma fonte com capacidades específicas, permitindo maior versatilidade

Figura 6 – Módulo Driver Ponte H - L298N



Fonte: RoboCore (2026)

4.1.5 Baterias de lítio

A alimentação é composta por quatro baterias de lítio recarregáveis de tensão 4,2V e capacidade 7800mAh, que fornecem energia para os motores, microcontrolador e sensores (figura 7). As baterias são carregadas por um carregador duplo bivolt observado também na figura 7, compatível com baterias do tipo Li-ion de 3,7 a 4,2 V, equipado com LEDs indicadores de carga e proteções contra curto-circuito e sobrecarga, garantindo recarga segura e simultânea de duas unidades.

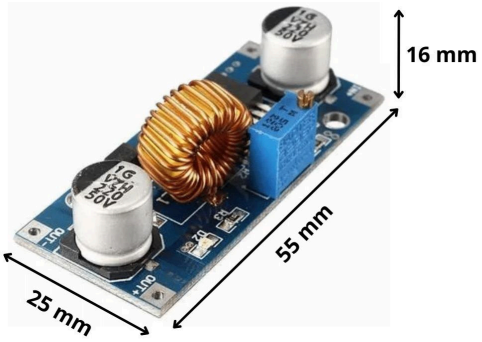
Figura 7 – Kit Carregador Duplo + 4 Baterias 18650 3,7v 7800mah

Fonte: Mercado livre (2026)

4.1.6 Regulador de tensão

O robô utiliza um módulo regulador de tensão ajustável, do tipo conversor DC-DC, ilustrado na figura 8, cuja função é adequar a tensão da fonte de alimentação aos níveis necessários para os componentes eletrônicos. No GMR, a tensão vinda das baterias é regulada para 5V. A saída do regulador alimenta o Raspberry Pi e o circuito de carga da ponte H, que alimenta os motores.

Figura 8 – Módulo controlador de tensão



Fonte: Mercado livre (2026)

4.1.7 Demais componentes

Demais componentes como rodas, parafusos, roda boba, botão interruptor e a placa de acrílico (figura 9) fazem parte de um kit de montagem de robô seguidor de linha. Também foram usados *jumpers* (condutores elétricos comumente usados em eletrônica) e uma protoboard para conexões de energia entre os componentes que requerem alimentação elétrica.

Figura 9 – estrutura de acrílico e roda boba

Fonte: Tech Sul Eletrônicos (2026)

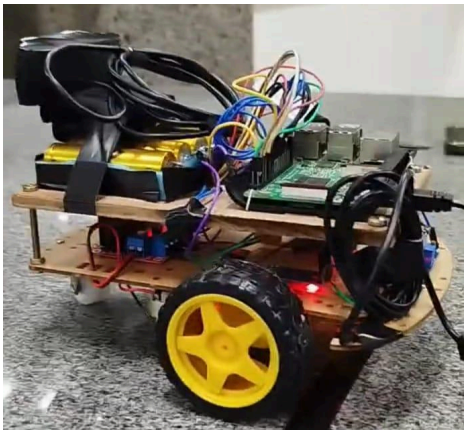
4.2 Montagem

Conforme visualizado na figura 10, onde se dispõe a montagem final do projeto e de forma mais detalhada no vídeo 1, acessível por meio do link disponibilizado no Apêndice A, a montagem foi feita sob 2 plataformas. Na plataforma inferior encontram-se as rodas, os motores, a roda boba, a ponte H, os encoders, o regulador de tensão, a protoboard e o botão interruptor. Cabe mencionar que a protoboard poderia ser substituída pela soldagem dos cabos, porém optamos por organização e possibilidade de expansão, caso fosse necessário. Na plataforma superior, fixam-se o Raspberry, a câmera e o módulo de baterias. Os componentes foram fixados com fita dupla face de boa qualidade, e se mantiveram estáveis.

O módulo de baterias foi produzido em impressora 3D, sendo seu modelo acessível pelo Apêndice A, Item 1. A montagem de conexões elétricas foi feita à mão, sendo uma junção de série e paralelo, de forma a obter 8,4V e o dobro da capacidade convencional de carga.

O segundo suporte do GMR foi feito em uma placa de MDF, e fixado na parte inferior com o uso de porcas e parafusos. Foi necessário executar novas perfurações no acrílico (parte inferior), que suportou de forma firme as adaptações. Um furo feito com fresa no centro da placa abre passagem para os *jumpers* que acessam os demais componentes.

Figura 10 – vista lateral GMR



Fonte: Do autor.(1)

4.3 Funcionamento

O GMR funciona locomovendo-se entre pontos predefinidos. Inicialmente, consideremos os pontos A e B para fins explicativos. O processo começa com a leitura de um QR Code, que representa um ponto ou localização conhecida. Considere que o robô está no ponto A inicialmente. No caso de exemplo, entre os pontos A e B existentes, seria lógico apresentar o QR relacionado ao ponto B. Ao ler o QR Code, o robô se desloca para o destino, nesse caso, o ponto B. Em todos os pontos de destino deve haver um QR de confirmação, que será lido pelo robô a fim de confirmar em qual ponto ele se localiza. Após a chegada ao ponto destino (nesse exemplo o ponto B), o robô espera a leitura de um QR que aponte para o novo ponto de destino, que, neste exemplo, deveria ser o ponto A. Essa movimentação pode ser observada no vídeo 1 (Apêndice A).

4.4 QR Codes

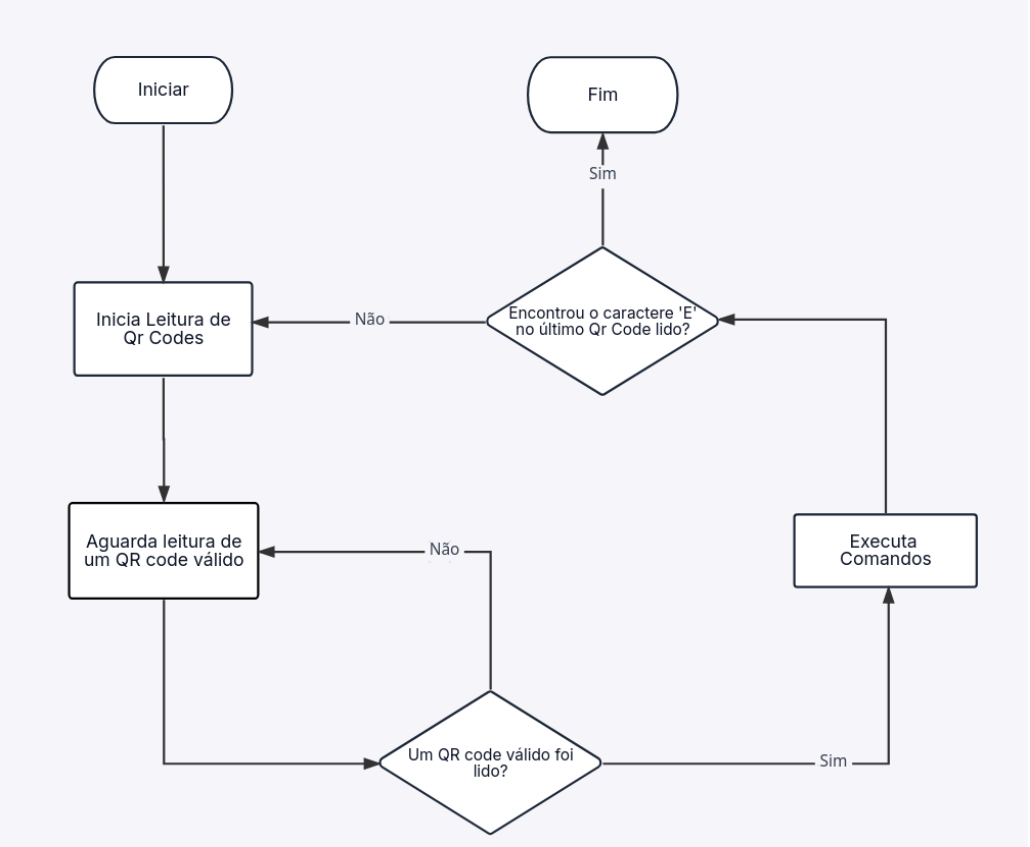
O conteúdo dos QRs é uma *string* (frase) escrita que segue uma estrutura padrão entendida pelo robô, detalhada na seção 4.6. Ao ler o QR e decodificar essa frase, o robô

executa os comandos identificados. Esses comandos guiam o GMR até o ponto de destino, ou seja, a rota é definida e calculada de forma manual, e o robô segue as instruções passadas As frases contidas nos QRs são escritas de forma manual, a fim de se adaptar às melhores rotas entre os pontos.

4.5 Fluxograma

A figura 11 apresenta o fluxograma da lógica de funcionamento do GMR. O processo se inicia aguardando a leitura de um QR Code e se repete até que seja lido um marcador válido. Após a leitura, inicia-se o processamento das instruções contidas no QR, ou seja, a movimentação do robô. Caso seja encontrado dentre os comandos de movimentação o comando de finalização (caractere “E”), o robô encerra o seu funcionamento, permanecendo parado até que seja reiniciado. Caso contrário, ao fim de sua movimentação, permanece parado à espera pela apresentação de um novo QR code.

Figura 11 – Fluxograma do funcionamento do sistema



Fonte: Do autor.(2)

4.6 Programação

O código completo utilizado na etapa de programação pode ser encontrado anexado no apêndice B, bem como no repositório do GitHub, apresentado no item 1 do referido apêndice.

A programação do GMR, escrita na linguagem C++, é responsável pela interface de comunicação entre os componentes e os atuadores, de acordo com a lógica de funcionamento do robô. Como observado no fluxograma, o funcionamento do GMR envolve a identificação de marcadores, processamento de dados e locomoção. Portanto, pode-se entender a programação como sendo um conjunto de 3 principais núcleos: da visão computacional, do processamento lógico e da atuação, descritos individualmente nas seções a seguir. Os núcleos conectam-se através da lógica da programação.

Inicialmente, foram desenvolvidas bibliotecas de programação exclusivas do projeto, contendo as funções relacionadas a cada núcleo. Entretanto, no projeto final, apenas a etapa de identificação dos marcadores, relacionada à visão computacional, foi implementada separadamente, sendo as demais etapas reunidas em um único arquivo.

4.6.1 Visão computacional

A visão computacional é responsável por identificar o conteúdo dos QR Codes, através de funções contidas na biblioteca OpenCV. A função *get_qrcode* presente no código principal executa o programa que busca os marcadores. O dado obtido da leitura correta de um marcador válido (aquele que apresenta dados na estrutura determinada para o funcionamento lógico do robô) é uma string que, após ser processada pela programação, fornece uma “lista” de comandos de movimentação.

4.6.2 Processamento lógico

O processamento lógico é o que conecta logicamente os núcleos do robô. É onde define-se quando e como chamar cada função. As funções precisam interagir entre si, por serem partes individuais de um todo, e essa interação é feita de maneira lógica e planejada. O programa relacionado à visão computacional é requisitado por esse núcleo, bem como as funções relacionadas à movimentação.

4.6.3 Movimentação

Mover o robô envolve informações de entrada e informações de saída. O que permite receber e enviar informações relacionadas à movimentação é a GPIO (General Purpose Input/Output) do Raspberry PI, manipulada pela biblioteca PIGPIO. Através dela é possível habilitar a GPIO a receber dados de sensores, bem como enviar dados através da manipulação da tensão fornecida em cada pino. Para mover o robô para frente, por exemplo, é necessário fornecer tensão elétrica pelo pino diretamente conectado ao driver dos motores, fazendo o driver permitir a alimentação dos motores, e, consequentemente, o movimento.

A movimentação do GMR é guiada por funções locais, criadas particularmente para o projeto: a função *forward* e a função *spin*. A função *forward* tem seu parâmetro dado em centímetros, ou seja, se o número passado para forward for 30, o robô se moverá 30 centímetros para frente. A função spin tem seu parâmetro dado em graus. Existe também a função *spin_by_ticks*, que pode substituir, caso necessário, a função spin, tendo seu parâmetro dado considerando a contagem de pulsos (ticks) do encoder. Girar 29 ticks é o equivalente a girar 90 graus. No programa final utilizamos ticks a fim de conseguir a melhor precisão possível nos testes, porém, considerar o uso da função spin tem o mesmo efeito didático no entendimento do funcionamento.

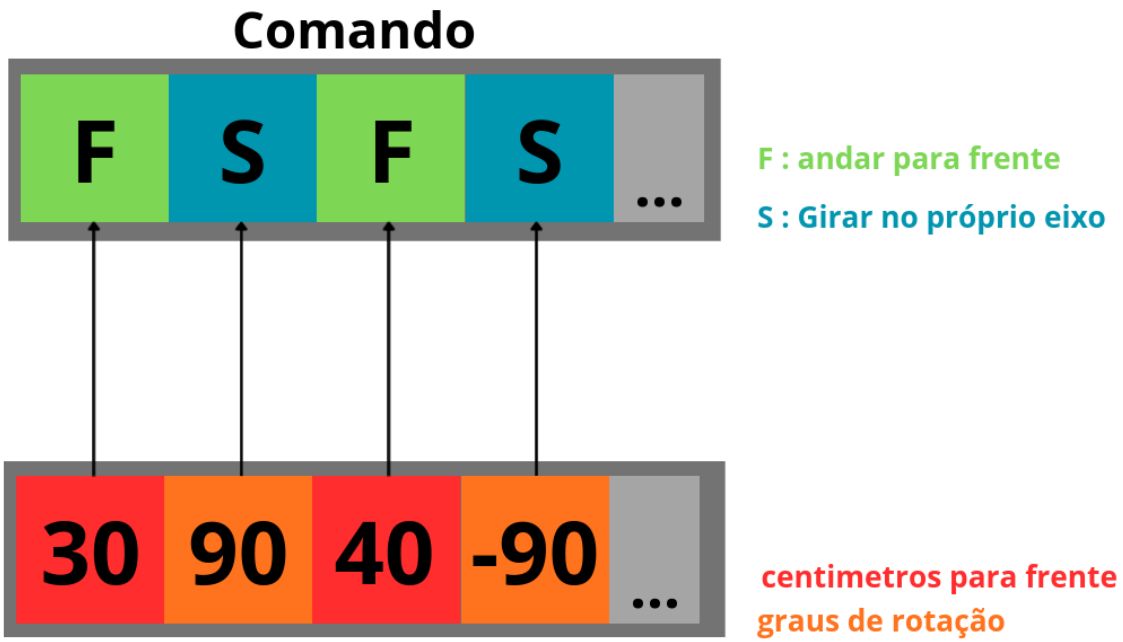
O encoder incrementa sua contagem a cada tick. Um tick é uma variação no nível lógico do pino, decorrente da variação da recepção da luz do sensor IR. Incrementar a contagem está relacionado, portanto, à rotação do eixo juntamente à roda, e, consequentemente, relacionado ao deslocamento do robô. Ou seja, um tick contado está relacionado a percorrer uma distância fisicamente, considerando a aderência ideal da roda ao chão, de forma a não deslizar. Tendo em mente que os ticks estão relacionados a uma distância real percorrida pela roda, para que fosse possível usar a função spin em graus e a forward em centímetros houve a necessidade de conversão de valores. Foram realizadas medições na roda, e seu diâmetro calculado foi de aproximadamente 20 cm. As 20 seções do disco do encoder, sendo consideradas as alterações de nível lógico nas bordas de subida e descida, resultam em 40 ticks a cada volta do disco. Constata-se, por fim, que percorrer 2 ticks corresponde a aproximadamente 1 cm. Para determinar quantos graus correspondem a 1 tick, foram feitos testes de forma empírica, utilizando códigos de teste que permitem a execução das funções de rotação de forma adaptada para teste, a fim de descobrir quantos ticks correspondiam a 1 volta completa. Como resultado, 1 tick em rotação corresponde a um

deslocamento angular de aproximadamente 3 graus.

4.6.3.1 Funcionamento da string

Como mencionado previamente, o conteúdo dos QRs é uma string. A string esperada para ser processada pelo robô segue um padrão. Para entender este padrão, adotemos as letras "F" para a função *forward* e "S" para a função *spin*, e observemos a figura abaixo.

Figura 12 – Representação da string de movimento



Fonte: Do autor.(3)

Consideremos também "parâmetro" como uma "quantidade" ou número passado para uma determinada função, sendo o parâmetro de F dado em centímetros, e o parâmetro de S dado em graus. Valores positivos para spin fazem com que o GMR gire para a direita, enquanto valores negativos o fazem girar para a esquerda. A estrutura da string segue o seguinte padrão:

F,S,F,S,F,S (…)

E deve ser escrita dessa forma ao fabricar o QR Code, respeitando a lógica de

funcionamento ao substituir as letras pelos parâmetros. O caractere vírgula deve fazer parte da string, separando cada parâmetro.

Ao produzir o QR-Code, as posições onde se encontram as letras F e S devem ser substituídas com os valores que se deseja passar para cada função. No exemplo da figura (12), a primeira movimentação do robô seria deslocar-se 30 centímetros para frente, girar 90 graus, andar mais 40 centímetros para frente e girar -90 graus. As posições são fixas, portanto, o primeiro parâmetro é sempre passado à função F. A sequência de comandos pode ser estendida em uma quantidade indeterminada, de forma que permita o robô chegar ao destino desejado.

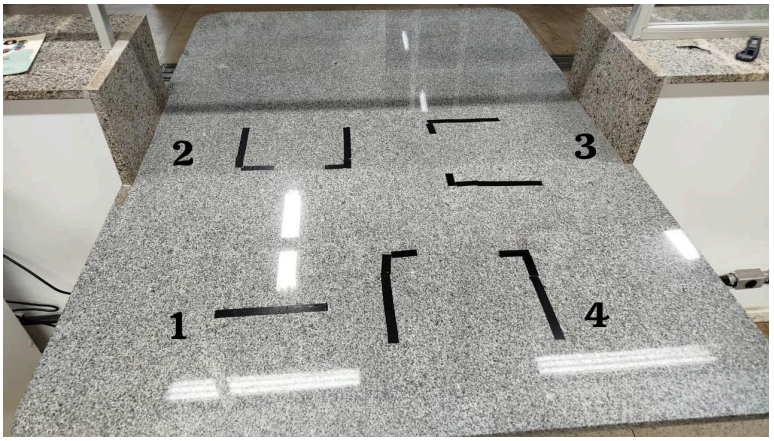
O caractere vírgula (,), na lógica do código, é utilizado como *token* para a função que transforma a string em um vetor. Esse vetor é passado para a função *processarComandos*, que é responsável por chamar as funções F e S alternadamente, passando como parâmetros os elementos do vetor.

4.6.3.2 Caractere de fim de execução

Ao executar todos os comandos contidos na string, existem duas possibilidades. No primeiro caso, onde o elemento final da string é um parâmetro, como visto na figura (12), ao fim da movimentação o robô reinicia o ciclo lógico de funcionamento aguardando um QR-Code. No segundo caso, se a string termina com o caractere ‘E’, como por exemplo na string “30,90,20,180,40,E”, o funcionamento do robô é cessado. O Raspberry continua ativo, porém o programa que sustenta o seu funcionamento é encerrado, podendo ser reiniciado e possibilitando o funcionamento normal do robô.

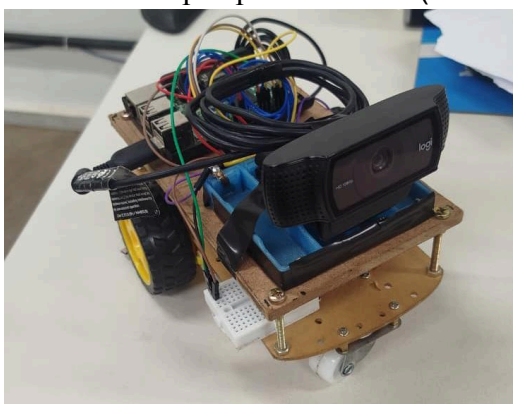
5 RESULTADOS

Figura 13 – Imagem dos setores para o teste realizado



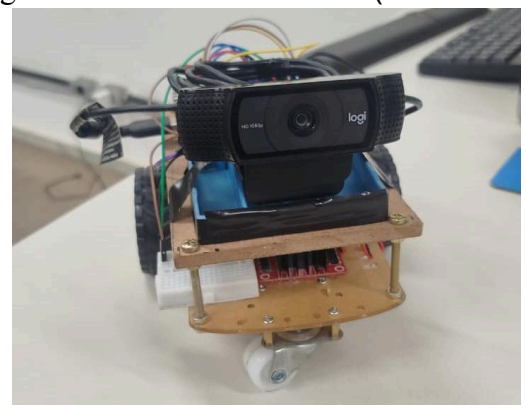
Fonte: Do autor.(4)

Figura 14 – vista em perspectiva GMR (SEM BATERIAS)



Fonte: Do autor.(5)

Figura 15 – vista frontal GMR (SEM BATERIAS)



Fonte: Do autor.(6)

A figura 13 apresenta uma simulação de um ambiente de circulação do robô. Os locais 2, 3 e 4 foram determinados como locais aos quais poderiam ser destino do robô. Na simulação do funcionamento, o robô vai do ponto 1 ao 2, do 2 ao 3, e do 3 ao 4. Os comandos que compõem a rota para locomover-se até esses pontos foram definidos através de medidas das distâncias. Por exemplo: o ponto 3 fica a 20 centímetros do ponto 2. Mover o GMR do ponto 2 ao ponto 3 deveria ser, nesse caso, "0,90,20", pois o ponto 3 se encontra exatamente à direita do ponto 2.

As movimentações corretas dependem da orientação do robô, portanto, é necessário uma maneira de identificar para qual direção o robô está orientado. Devido à limitações do projeto, adicionamos uma etapa de verificação de posição e orientação ao código principal. Após o fim da execução de uma sequência de comandos, o GMR inicia o processo de leitura de QR-Codes, que, neste momento, serve para tentar ler um marcador que estaria fixo no ponto destino. Ler este marcador indica ao robô o lugar onde ele chegou, e para qual direção está orientado.

Não há uma forma ideal de interação do GMR com o ambiente, o que implica em não ser possível assegurar a chegada ao ponto destino corretamente. Isso se deve ao fato de que formas de integração com o ambiente ao redor apresentaram aumentos expressivos no custo e/ou na complexidade de desenvolvimento, o que foge da proposta de ser uma planta didática para aprendizagem a nível técnico e de custo relativamente baixo, porém impacta diretamente a precisão da movimentação.

A precisão das medições através do encoder se mostraram vulneráveis e pouco suficientes. A razão entre a quantidade de ticks e a distância percorrida foi baixa, o que dificulta o controle. Além disso, por serem compostos de elementos individuais e externos, diferentemente de um motor com encoder, os encoders são submetidos à vibrações enquanto o robô se movimenta, podendo causar alterações nas medições, e, consequentemente, na movimentação. Outro fator que altera a precisão na movimentação foi a folga entre as peças que fixam o motor ao chassi, e a própria folga interna do motor.

Ao analisar o vídeo 1 do Apêndice A, constata-se que, apesar dos fatores que prejudicam a movimentação precisa do robô, ele foi capaz de ler corretamente os marcadores e se deslocar entre os pontos propostos.

A leitura de QR Codes funcionou bem e sem falhas, bem como a interpretação e processamento dos dados obtidos.

A Tabela a seguir apresenta a estimativa de custos do projeto, considerando a necessidade de aquisição das peças inicialmente emprestadas, sendo o total gasto R$984,59.

Na seção referências é possível encontrar as fontes de aquisição desses materiais.

Tabela 1 – Custos estimados do projeto

Produto	Preço(R$)	Forma de aquisição	Quantidade
Câmera Logitech C920	349,90	Emprestado	1
Kit Carregador Duplo+4 Baterias 18650 4,2V 7800mah	73,59	Comprado	1
Raspberry Pi 3 model B	480,16	Emprestado	1
Kit seguidor de linha	51,94	Emprestado	1
Regulador de tensão XL4015	14,00	Comprado	1
Compartimento de baterias	10,00	Comprado	1
Mini protoboard	5,00	Emprestado	1

Fonte: Do autor.(7)

6 CONSIDERAÇÕES FINAIS

Sob a ótica da planta didática, o desenvolvimento e estudo do projeto possibilitou expressivamente o aprendizado nas áreas da robótica, programação, eletrônica, projeto estrutural e controle. Na robótica, destacam-se conceitos fundamentais como tração diferencial, odometria e navegação autônoma baseada em marcadores visuais. Na programação, o projeto proporcionou a leitura e interpretação de dados de sensores, estruturação de código em módulos, uso de bibliotecas externas como OpenCV e PIGPIO, manipulação de strings, lógica de controle e integração entre software e hardware, além de desenvolver a capacidade de raciocínio lógico. Na eletrônica, foram trabalhados conceitos como controle de motores DC por PWM, funcionamento de pontes H, encoders ópticos, regulação de tensão e gerenciamento de baterias de lítio. No projeto estrutural, o

desenvolvimento do chassi em duas plataformas exigiu decisões sobre distribuição de componentes, fixação mecânica e adaptações como perfurações e impressão 3D. Por fim, na área de controle, o projeto introduziu na prática o conceito de controle de malha aberta e controle de movimentação por odometria. Estas áreas de conhecimento são em sua maioria abordadas no curso de Automação Industrial, o que torna o GMR uma forma conveniente de aprofundar e aprimorar o conhecimento e aprendizagem do curso.

Entretanto, a proposta de aplicação em uma plataforma transportadora de objetos não foi atendida, uma vez que a inclusão dessa funcionalidade demandaria a ampliação do escopo das atividades de desenvolvimento, exigindo novas adaptações e testes, o que não foi viável dentro do tempo disponível.

A possibilidade de transporte pode ser implementada mediante a adaptação de uma nova superfície no robô que seja capaz de suportar objetos, em conjunto à realização de testes direcionados a avaliar a capacidade máxima de carga. Dessa forma, será possível avaliar o peso máximo suportado e propor adaptações nos materiais estruturais, visando garantir estabilidade ao sistema sem interferir nos componentes nem comprometer a execução dos movimentos.

A estrutura projetada comportou com espaço e estabilidade suficiente os componentes do robô. Entretanto, os encoders podem sofrer inconsistências na leitura devido à vibração.

A implementação de um aplicativo de controle, prevista nos objetivos específicos, não foi concluída em razão das limitações de tempo impostas pelo escopo do projeto. A proposta consistia na criação de uma aplicação móvel desenvolvida com o MIT App Inventor 2, que permitiria ao usuário enviar comandos de movimentação ao robô por meio de comunicação sem fio, eliminando a necessidade de QR Codes pré-programados para determinadas operações. Para trabalhos futuros, recomenda-se a implementação dessa funcionalidade utilizando comunicação via Bluetooth ou Wi-Fi, já suportados nativamente pelo Raspberry Pi 3, o que tornaria o sistema mais flexível e interativo sem implicar aumento significativo de custo ou complexidade.

As informações apresentadas no capítulo de resultados validam a prova de conceito proposta, demonstrando a viabilidade do sistema. Ainda que o aplicativo não tenha sido implementado, o sistema atendeu parcialmente aos objetivos estabelecidos, evidenciando

potencial para futuras melhorias e ampliações.

Em relação aos objetivos específicos estabelecidos, é possível avaliá-los de forma individual. A leitura dos marcadores visuais por meio da biblioteca OpenCV foi implementada com sucesso, permitindo ao robô identificar os QR Codes e receber as instruções de movimentação. O transporte automatizado de objetos entre pontos foi parcialmente atendido: o robô demonstrou capacidade de deslocamento entre os pontos definidos, porém sem uma plataforma física dedicada ao transporte de cargas, o que limita a aplicação prática. A criação de um aplicativo de comando não foi concluída dentro do prazo disponível, constituindo uma pendência para trabalhos futuros. Por fim, o chassi foi projetado e construído com sucesso em duas plataformas, apresentando estabilidade estrutural suficiente para suportar os componentes do sistema.

A adoção de motores mais avançados tende a minimizar desvios de trajetória, uma vez que apresentam menor variação de desempenho entre unidades do mesmo modelo, problema comum em motores DC convencionais.

A utilização de baterias de maior qualidade contribuiria para a estabilidade da alimentação elétrica do sistema, reduzindo variações de tensão que afetam diretamente o funcionamento de motores e sensores.

No aspecto técnico, é necessário salientar que o método de navegação escolhido, da maneira em que foi aplicado, não é o método de locomoção autônoma mais preciso quando o foco é o transporte de objetos. Não há neste projeto uma maneira ideal de integração em tempo real com o ambiente, de forma a detectar e corrigir desvios de trajetória e obstáculos, e garantir que o robô chegue ao destino de forma assertiva. Todavia, é justo considerar as limitações no decorrer do desenvolvimento do projeto, bem como a proposta acadêmica do mesmo. As adaptações necessárias para sanar os problemas supracitados exigem uma mudança significativa no funcionamento do robô, aumentando de forma significativa a complexidade do projeto, e, principalmente, aumentando o custo. O robô foi desenvolvido com base em materiais relativamente mais acessíveis, facilitando a ampla implementação e proporcionando uma maneira de se aprender conceitos comuns no mundo da robótica. A plataforma pode ser utilizada como recurso didático em diferentes contextos, como laboratórios de ensino e ambientes de aprendizagem, contribuindo para a compreensão de conceitos relacionados à automação, robótica e sistemas embarcados.

REFERÊNCIAS

AMAZON. Logitech Webcam HD Pro C920. Disponível em: <https://a.co/d/0gJhuEMr> .Acesso em: 3 fev. 2026.

CONCEIÇÃO, Gabriel Carvalho Domingos da; COLOMBO, Letícia Pedroso; FERREIRA, Lucas Mota. Empilhadeira robótica autônoma: robô em miniatura. 2019. Trabalho de Conclusão de Curso (Técnico Integrado em Automação Industrial) – Instituto Federal de Minas Gerais, Campus Avançado Itabirito, Itabirito, 2019.

CUNNINGHAM, Drew. Automated material handling: optimizing logistics with robotics and software.AutoStore,7 jan. 2026. Disponível em: <https://www.autostoresystem.com/insights/an-easy-guide-to-aut omated-material-handling>. Acesso em: 13 jan. 2026.

DAY, Lance; MCNEIL, Ian (eds.). *Biographical Dictionary of the History of Technology*. London; New York: Routledge, 1996. Disponível em: <https://books.google.co.nz/books?id=m8TsygLyfSMC&pg=PA78&lpg=PA78&dq=patent+barsanti+matteucci+1857>. Acesso em: 3 fev. 2026.

GLOBALSPEC**.** DC Motors. Disponível em: <https://www.globalspec.com/reference/8270/348308/3-8-1-dc-motors>. Acesso em: 3 fev. 2026.

IBM. Cenários de Telemetria: Radio Frequency Identification (RFID). 2018. Última atualização: 7 dez. 2018. Disponível em: <https://www.ibm.com/docs/pt-br/ibm-mq/7.5.0?topic=ssfksj-7-5-0-com-ibm-mq-pro-doc-q002800--htm>. Acesso em: 3 fev. 2026.

IBM. O que é LiDAR. 2022. Disponível em:<https://www.ibm.com/br-pt/think/topics/lidar> .Acesso em: 3 fev. 2026.

KIT CHASSI ROBÔ 2WD ACRÍLICO COM MOTORES E ENCODERS – MINGWU ELECTRONICS. Disponível em: <https://www.mercadolivre.com.br/kit-chassi-rob-2wd-acrilico-com-motores-e-encoders-mingwu-electronics/p/MLB34379705>. Acesso em: 13 jan. 2026.

LEE, Jeisung; HYUN, Chang-Ho; PARK, Mignon. A vision-based automated guided vehicle system with marker recognition for indoor use. Sensors, Basel, v. 13, n. 8, p. 10052-10073, 7 ago. 2013. Disponível em: <https://www.mdpi.com/1424-8220/13/8/10052>. Acesso em: 13 jan. 2026.

MERCADO LIVRE. Conversor de tensão DC Step-Down abaixa tensão XL4015 5 A. Disponível em: <https://produto.mercadolivre.com.br/MLB-5551004346-conversor-tenso-dc-step-down-abaixa-tenso-xl4015-5a-_JM>. Acesso em: 3 fev. 2026.

MERCADO LIVRE. Disco codificador de disco codificado 10 pcs (20 contém sensor). Disponível em: <https://www.mercadolivre.com.br/codificador-de-disco-codificado-10pcs-20-contem-sensor-de/p/MLB2047353206>. Acesso em: 3 fev. 2026.

MERCADO LIVRE. Kit carregador duplo + 4 baterias 18650 3,7 V 7800 mAh. Disponível em:<https://produto.mercadolivre.com.br/MLB-3279300240-kit-carregador-duplo4-baterias-18650-37v-7800mah-_JM>. Acesso em: 3 fev. 2026.

MERCADO LIVRE. Módulo sensor velocidade encoder acoplador óptico Arduino. Disponível em: <https://www.mercadolivre.com.br/sensor-velocidade-encoder-acoplador-optico-arduino/up/MLBU2520747103>. Acesso em: 3 fev. 2026.

MERCADO LIVRE. Raspberry Microcontrolador Init I3 Pi 3 Mod B. Disponível em: <https://www.mercadolivre.com.br/raspberry-microcontrolador-init-i3-pi-3-mod-b/p/MLB2042158406>. Acesso em: 3 fev. 2026.

ROBOCORE. Driver Motor Ponte H L298N. Disponível em: <https://www.robocore.net/driver-motor/driver-motor-ponte-h-l298n>. Acesso em: 3 fev. 2026.

ROBOCORE. Motor DC 3-6V com caixa de redução e eixo duplo. Disponível em: <https://www.robocore.net/motor-caixas-de-reducao/motor-dc-3-6v-com-caixa-de-reducao-e-eixo-duplo>. Acesso em: 3 fev. 2026.

SANTOS, Miguel Henrique de Aguiar. Projeto – QR Guided Mobile Robot. 2026. Duração: 6:12 min. Vídeo gravado pelo grupo de desenvolvimento. Disponível em: <https://youtu.be/dRRSmKqsFWg>. Acesso em: 3 fev. 2026.

SNATUZ. QR-Code-Guided-Robot. GitHub, 2026. Disponível em: <https://github.com/Snatuz/QR-Code-Guided-Robot>. Acesso em: 28 de março de 2026.

SEMBORSKI, Jakub; IDZKOWSKI, Adam. A review on positioning techniques of mobile robots. Robotic Systems and Applications, v. 4, n. 1, p. 30–43, 17 fev. 2024. Disponível em: <https://www.extrica.com/article/23893>. Acesso em: 13 jan. 2026.

TECHSUL ELETRÔNICOS. Kit chassi acrílico para robô 2WD. Disponível em: <https://techsuleletronicos.com.br/product/kit-chassi-acrilico-p-robo-2wd/>. Acesso em: 3 fev. 2026.

THINGIVERSE. Modelo 3D de suporte de baterias. Disponível em: <https://www.thingiverse.com/thing:2339441>. Acesso em: 3 fev. 2026.

UNIVERSAL ROBOTS. A história da robótica: dos autômatos antigos aos cobots e outros robôs modernos. Universal Robots Brasil, 17 nov. 2023. Disponível em: <https://www.universal-robots.com/br/blog/a-historia-da-robotica-dos-automatos-antigos-aos-cobots-e-outros-robos-modernos/>. Acesso em: 13 jan. 2026.

VIEIRA, Rodrigo; SILVA, Dino; RIBEIRO, Eliseu; PERDIGOTO, Luís; COELHO, Paulo Jorge. Performance evaluation of computer vision algorithms in a programmable logic controller: an industrial case study. Sensors, Basel, v. 24, n. 3, p. 843, 28 jan. 2024. Disponível em: <https://www.mdpi.com/1424-8220/24/3/843>. Acesso em: 13 jan. 2026.

WANG, Aileen**.** *How Robots Are Used in Transportation and Transform the Industry.* Fdata Robot, 2025. Disponível em: <https://www.fdatabot.com/pt/how-robots-are-used-in-transportation-and-transform-the-industry/>. Acesso em: 3 fev. 2026.

APÊNDICE A - ESTRUTURA E FUNCIONAMENTO

Item 1 – Arquivo do Modelo 3D do Suporte de Baterias

Arquivo utilizado para impressão 3D do suporte de baterias do robô

Disponível em: <Flexing Battery Holders with Integrated Spring by Pixelle - Thingiverse>.

Item 2 – Vídeo 1 - Estrutura e Funcionamento

Vídeo demonstrando a montagem final do projeto e seu funcionamento.

Disponível em: <https://youtu.be/dRRSmKqsFWg>.

APÊNDICE B - CÓDIGO-FONTE E REPOSITÓRIO DO PROJETO

O repositório do projeto no GitHub, que disponibiliza informações adicionais, como a programação, as concepções iniciais e os arquivos utilizados no desenvolvimento da plataforma, encontra-se disponível a seguir. Na sequência, apresenta-se o código-base do robô.

Item 1 – Repositório do projeto:
Disponível em: <https://github.com/Snatuz/QR-Code-Guided-Robot>

Item 2 - Código-fonte do robô:

2.1 - Main:

```
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <pigpio.h>
#include <sstream>
#include <unistd.h>
#include <vector>

//      ORGANIZADO E DOCUMENTADO POR IA, FEITO POR MIGUEL (SNATUZ).
//-------------------------------------------------------------------------------//
// MAPEAMENTO DE PINOS GPIO

#define pinencoder_esq 7
```

```
#define pinencoder_dir 8
#define IN1 2   // Motor esquerdo
#define IN2 3   // Motor esquerdo
#define IN3 17  // Motor direito
#define IN4 27  // Motor direito
#define EN1 4   // PWM motor esquerdo
#define EN2 22  // PWM motor direito

//-------------------------------------------------------------------------------//
// PROTOTIPOS DE FUNÇÕES

int spin(int angle);
void callback_esq(int gpio, int level, uint32_t tick);
void callback_dir(int gpio, int level, uint32_t tick);
void spin_start();
void stop();
void forward(int distance);
int spin_by_ticks(int ticks);
std::vector<std::string> splitString(const std::string &str, char delimiter = ',');
void processarComandos(const std::vector<std::string> &comandos);
int QR_search();
std::string get_qrcode();

//-------------------------------------------------------------------------------//
// VARIÁVEIS GLOBAIS

int distance_counter = 0;
std::string qr_code_data;

//-------------------------------------------------------------------------------//
// STRUCT DO MOTOR

struct motor {
   unsigned int PWM;
```

```
    unsigned int previous_PWM = 0;
    unsigned int velocity;
    unsigned int encoder_counter = 0;
    unsigned int previous_encoder_counter = 0;
    float error;
    float previous_error;
    float correction;
    float derivative;
    float integral;
} typedef motor;

motor l_motor;
motor r_motor;
motor *left_motor  = &l_motor;
motor *right_motor = &r_motor;

// Constantes PID (reservadas para implementação futura)
float KP = 0;
float KI = 0;
float KD = 0;
float setpoint = 0;

//-------------------------------------------------------------------------------//
// FUNÇÕES

std::string get_qrcode() {
    system("./qrcode_reader");

    std::ifstream arquivo("qrcode.txt");
    std::string buffer;
    std::getline(arquivo, buffer);

    return (buffer == "empty") ? "empty" : buffer;
}
```

```
void callback_esq(int gpio, int level, uint32_t tick) {
    left_motor->encoder_counter++;
}

void callback_dir(int gpio, int level, uint32_t tick) {
    right_motor->encoder_counter++;
    distance_counter++;
}

void stop() {
    gpioWrite(EN1, 0);
    gpioWrite(EN2, 0);
    gpioPWM(IN1, 0);
    gpioPWM(IN2, 0);
    gpioPWM(IN3, 0);
    gpioPWM(IN4, 0);
}

void forward(int distance) {
    stop();
    gpioWrite(IN1, 1);
    gpioWrite(IN3, 1);
    gpioPWM(EN1, 900);
    gpioPWM(EN2, 920); // Valores calibrados empiricamente para andar reto

    distance_counter = 0;
    while (distance_counter < distance) {
        usleep(50000);
    }
    stop();
}

void spin_start() {
```

```
    gpioPWM(EN2, 900);
}

int spin(int angle) {
    stop();

    if (angle > 0) {
        gpioWrite(IN3, 0);
        gpioWrite(IN4, 1);
    } else {
        gpioWrite(IN3, 1);
        gpioWrite(IN4, 0);
    }

    if ((angle > 360 && angle != 999) || (angle < -360 && angle != -999)) {
        std::cout << "\nAngulo invalido. Comando ignorado. Prosseguindo...\n";
        return 0;
    }

    if (angle == 999 || angle == -999) {
        spin_start();
        return 0;
    }

    angle = (angle < 0) ? -angle : angle;

    spin_start();

    distance_counter = 0;
    while (distance_counter < (angle * 0.3)) {
        usleep(50000);
    }

    stop();
```

```
    return 0;
}

int spin_by_ticks(int ticks) {
    stop();

    if (ticks > 0) {
        gpioWrite(IN1, 1);
        gpioWrite(IN2, 0);
        gpioWrite(IN3, 0);
        gpioWrite(IN4, 1);
    } else {
        gpioWrite(IN1, 0);
        gpioWrite(IN2, 1);
        gpioWrite(IN3, 1);
        gpioWrite(IN4, 0);
    }

    ticks = (ticks < 0) ? -ticks : ticks;

    spin_start();

    distance_counter = 0;
    while (distance_counter < ticks) {
        usleep(100000);
    }

    stop();
    return 0;
}

std::vector<std::string> splitString(const std::string &str, char delimiter) {
    std::vector<std::string> tokens;
    std::stringstream ss(str);
```

```
    std::string token;

    while (std::getline(ss, token, delimiter)) {
        if (!token.empty()) {
            tokens.push_back(token);
        }
    }
    return tokens;
}

void processarComandos(const std::vector<std::string> &comandos) {
    std::cout << "Iniciando processamento de comandos..." << std::endl;

    bool chamarFuncaoF = true;

    for (const std::string &cmd : comandos) {

        if (cmd == "E") {
            std::cout << "Comando 'E' encontrado. Encerrando." << std::endl;
            exit(0);
        }

        int valorNumerico;
        try {
            valorNumerico = std::stoi(cmd);
        } catch (const std::invalid_argument &e) {
            std::cerr << "ERRO: '" << cmd << "' nao e um numero valido. Pulando." << std::endl;
            continue;
        }

        if (chamarFuncaoF) {
            forward(valorNumerico);
        } else {
            spin_by_ticks(valorNumerico);
```

```
        }

        usleep(500000);
        chamarFuncaoF = !chamarFuncaoF;
    }

    std::cout << "Processamento de comandos concluido." << std::endl;
}

int QR_search() {
    qr_code_data = "empty";
    distance_counter = 0;

    while (qr_code_data == "empty") {
        qr_code_data = get_qrcode();
    }

    std::cout << "\nQR Code lido: " << qr_code_data << std::endl;

    stop();
    usleep(1000000);

    std::vector<std::string> comandos = splitString(qr_code_data);
    processarComandos(comandos);

    return 0;
}

//-------------------------------------------------------------------------------//
// MAIN

int main() {
    if (gpioInitialise() < 0) {
        std::cerr << "Falha ao inicializar pigpio." << std::endl;
```

```
        return 1;
    }

    gpioSetPWMrange(EN1, 1000);
    gpioSetPWMrange(EN2, 1000);

    gpioSetMode(pinencoder_esq, PI_INPUT);
    gpioSetMode(pinencoder_dir, PI_INPUT);
    gpioSetMode(IN1, PI_OUTPUT);
    gpioSetMode(IN2, PI_OUTPUT);
    gpioSetMode(IN3, PI_OUTPUT);
    gpioSetMode(IN4, PI_OUTPUT);
    gpioSetMode(EN1, PI_OUTPUT);
    gpioSetMode(EN2, PI_OUTPUT);

    gpioSetPullUpDown(pinencoder_esq, PI_PUD_UP);
    gpioSetPullUpDown(pinencoder_dir, PI_PUD_UP);

    gpioSetAlertFunc(pinencoder_esq, callback_esq);
    gpioSetAlertFunc(pinencoder_dir, callback_dir);

    usleep(500000);

    while (1) {
        stop();
        QR_search();
        get_qrcode(); // Leitura do QR de confirmação de chegada
    }

    gpioTerminate();
    return 0;
}
```

2.2 qrcode_reader:

```
#include <fstream>
#include <iostream>
#include <opencv2/core.hpp>
#include <opencv2/imgproc.hpp>
#include <opencv2/objdetect.hpp>
#include <opencv2/videoio.hpp>
#include <string>
#include <unistd.h>

using namespace cv;

VideoCapture cam(0); // 0 = câmera padrão do sistema

int save_code(std::string code);

std::string get_qrcode() {
    QRCodeDetector qr;
    Mat frame;

    cam >> frame;

    if (frame.empty()) {
        return "empty";
    }
```

```
    std::vector<Point> bbox;
    std::string data = qr.detectAndDecode(frame, bbox);

    return data.empty() ? "empty" : data;
}

int save_code(std::string code) {
    std::ofstream arquivo("qrcode.txt");

    if (!arquivo.is_open()) {
        return 1;
    }

    arquivo << code;
    arquivo.close();
    return 0;
}

int main() {
    if (!cam.isOpened()) {
        std::cerr << "Erro: nao foi possivel abrir a camera." << std::endl;
        return 1;
    }
```

```
    std::cout << "Camera aberta. Aguardando QR Code..." << std::endl;

    while (true) {
        std::string qr_data = get_qrcode();

        if (qr_data != "empty") {
            std::cout << "QR Code lido: " << qr_data << std::endl;

            if (save_code(qr_data) != 0) {
                std::cerr << "Erro ao salvar o arquivo." << std::endl;
            }
            break;
        }
    }

    cam.release();
    return 0;
}
```